

CS 3006

Parallel and Distributed Computing

Lecture 5

Danyal Farhat

FAST School of Computing

NUCES Lahore

Network Topologies for Parallel Architectures, and Evaluation of Static Inter-connections

Lecture's Agenda

- **Static Interconnection vs Dynamic Interconnections**
- **Basic Interconnections**
 - **Bus**
 - **Crossbar Network**
 - **Multistage Omega Network**
 - **Linear Arrays**
 - **K-d Meshes**
 - **Hypercube**
 - **Tree Based Networks**

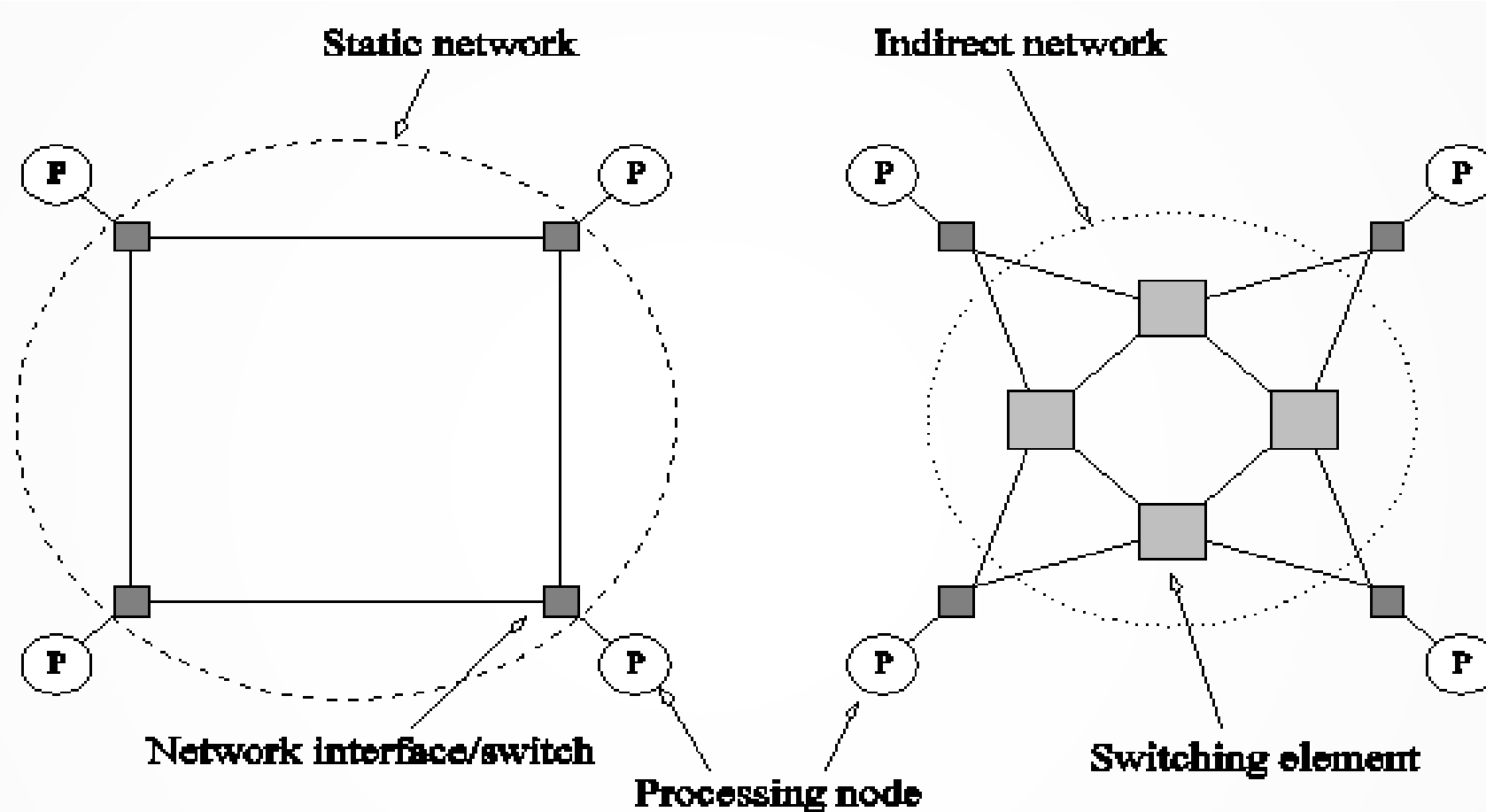
Lecture's Agenda (Cont.)

- **Evaluating Static Interconnections**
 - **Cost**
 - **Diameter**
 - **Bisection Width**
 - **Arc Connectivity**
- **Summary**
- **Additional Resources**

Static vs Dynamic Interconnections

- Interconnection networks **carry data** between processors and to memory
- Interconnects are **made** of processing elements, switches and links (wires, fiber) etc.
- Interconnects are **classified** as static or dynamic
- Static networks consist of **point-to-point** communication links among processing nodes and are also referred to as **direct** networks
- Dynamic networks are **built using switches** and communication links
- Dynamic networks are also referred to as **indirect** networks

Static vs Dynamic Interconnections (Cont.)



Classification of interconnection networks: (a) a static network; and (b) a dynamic network.

Shared Memory Interconnection Network

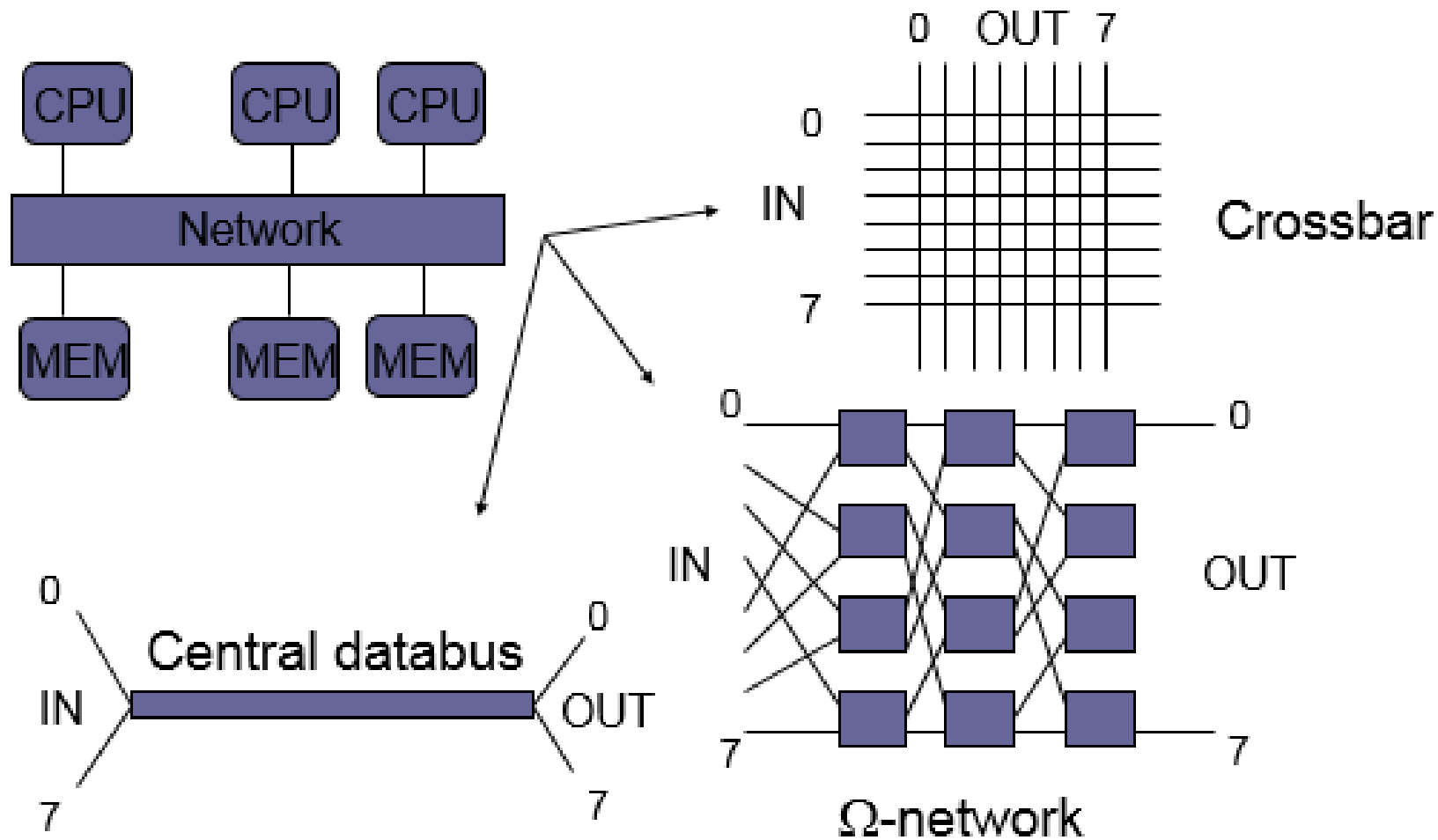
Problem: How to **interconnect** CPUs with each other and to the memory

Solution: Using network topologies.

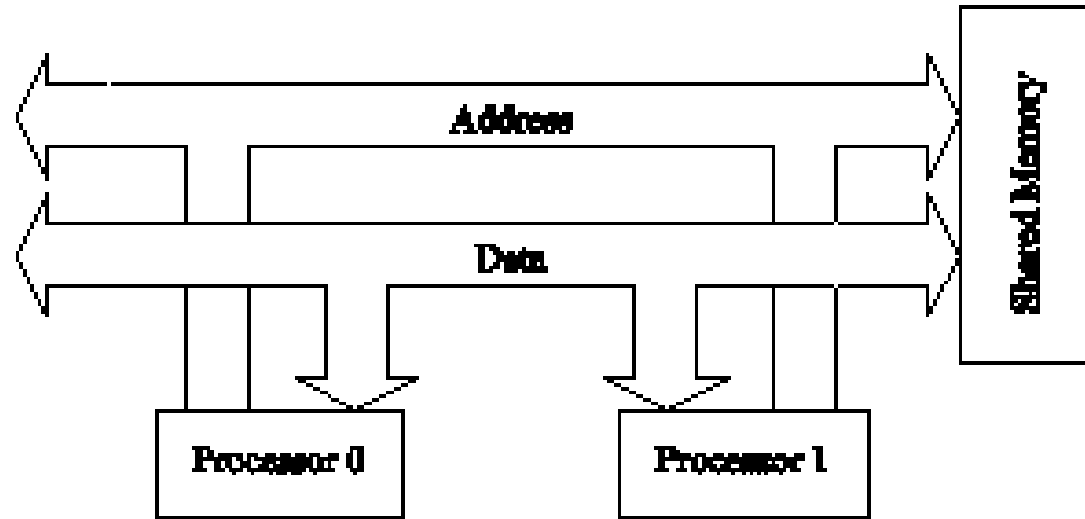
Three main topologies are:

- Central databus (**$O(1)$** connections - n shared)
- Crossbar (**$n \times n$** connections - datapath without sharing)
- Ω -network (**$n \log n$** connections - $\log n$ switching stages and shared on a path)

Shared Memory Interconnection Network (Cont.)

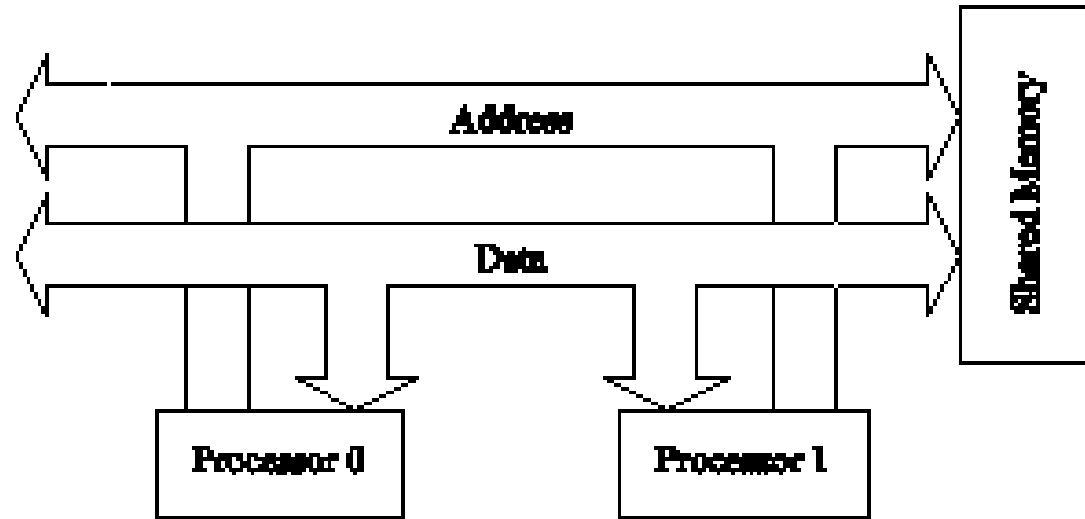


Bus



- Simplest and **earliest** parallel machines used buses
- All processors access a **common bus** for exchanging data
- The **distance** between any two nodes is $O(1)$ in a bus
- Bus provides a convenient **broadcast** media

Bus (Cont.)

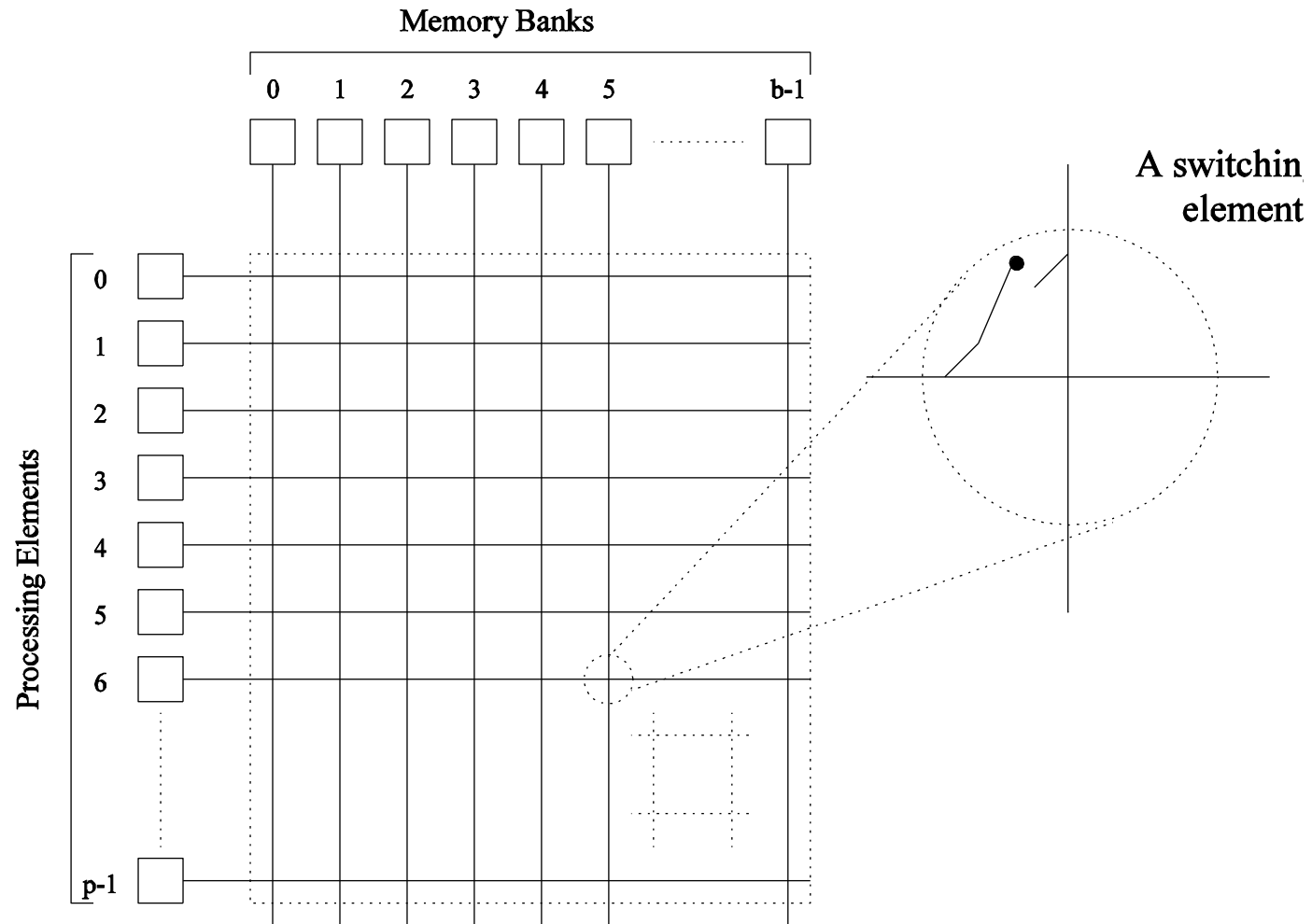


- The **bandwidth** of the shared bus is a major bottleneck
- Typical bus based machines are **limited** to dozens of nodes
- Sun Enterprise servers and Intel Pentium based shared-bus multiprocessors are **examples** of such architectures

Crossbar Network

- Crossbar network **allows** any system processor to connect to any other processor or memory unit
- A **simple way** of connection between processors (p) and memory banks (m)
- Used in the design of:
 - High performance small scale **multiprocessors**
 - Direct network **routers**
 - Large scale **indirect networks** as fundamental components

Crossbar Network (Cont.)



Crossbar Network (Cont.)

- Uses an $p \times m$ grid of switches to connect p inputs to m outputs in a non-blocking manner
- The **cost** of a crossbar of p processors grows as $O(p \times p)$
- This is generally **difficult to scale** for large values of p
- Examples of **machines** that employ crossbars include the Sun Ultra HPC 10000 and the Fujitsu VPP500
- Crossbars have excellent **performance scalability** but poor cost scalability
- Buses have excellent **cost scalability**, but poor performance scalability

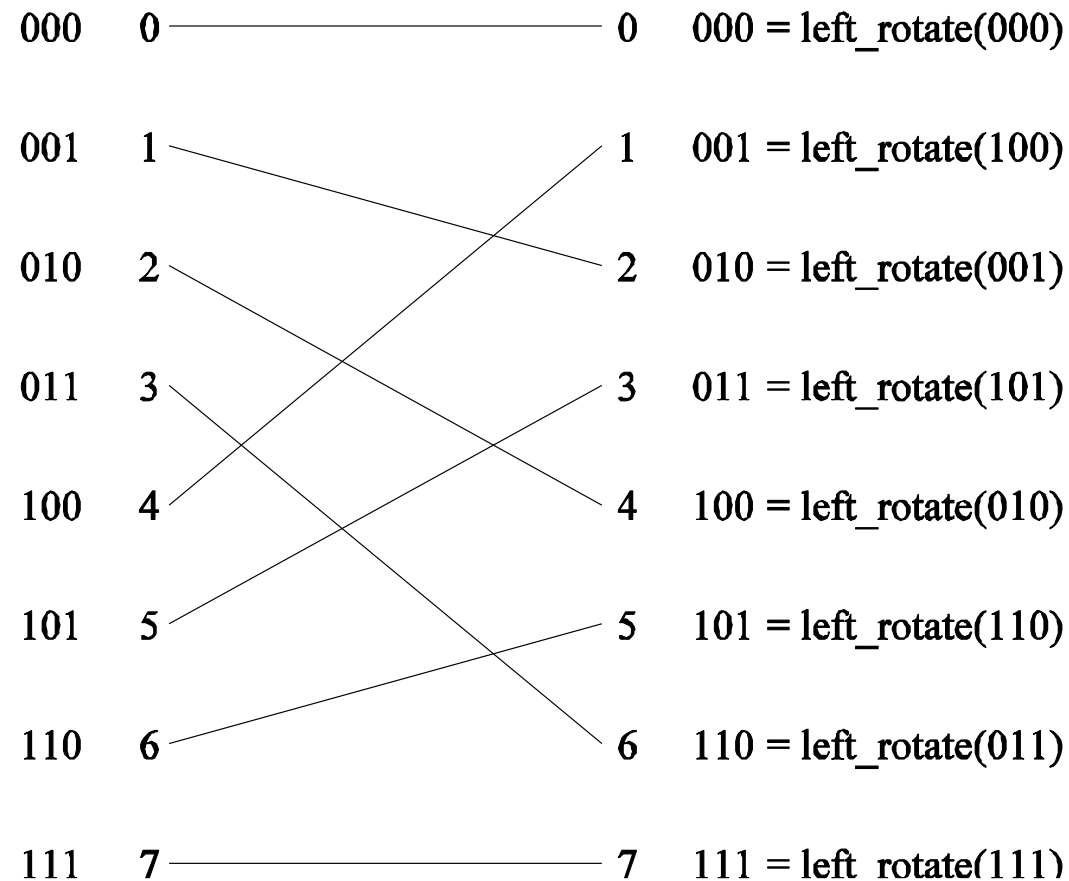
Multistage Omega Network

- One of the **most commonly used** multistage interconnects is the Omega network
- This network consists of **$\log p$ stages**, where p is the number of inputs/outputs
- At each stage, **input i is connected j**

$$j = \begin{cases} 2i, & 0 \leq i \leq p/2 - 1 \\ 2i + 1 - p, & p/2 \leq i \leq p - 1 \end{cases}$$

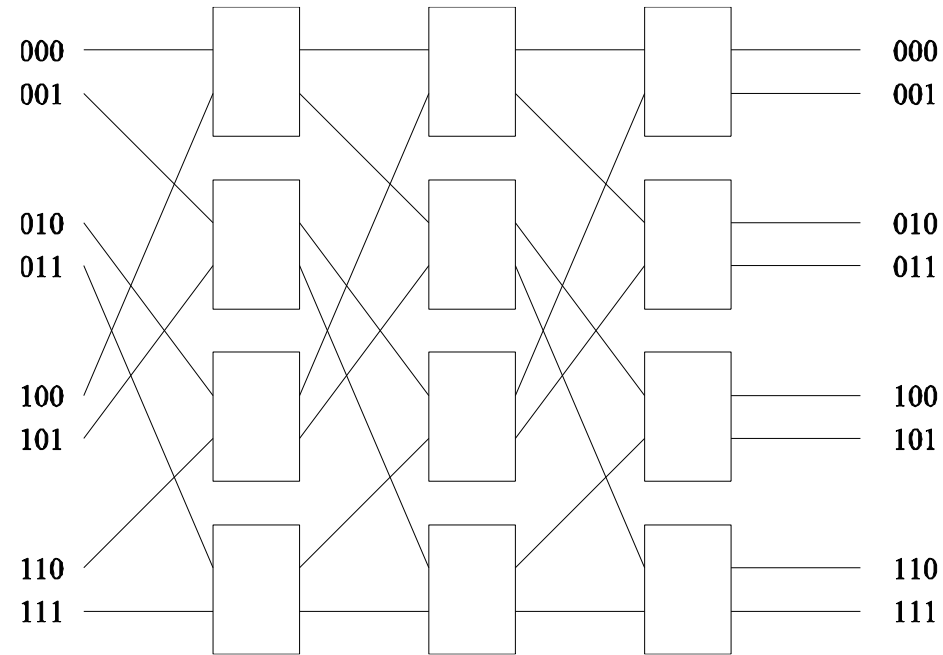
Multistage Omega Network (Cont.)

- Each stage of the Omega network implements a **perfect shuffle** as follows:



8 x 8 Omega network

- A complete Omega network with the perfect shuffle interconnects and switches can now be illustrated:



- An omega network has $p/2 \times \log p$ switching nodes, and the cost of such a network grows as $(p \log p)$

Multistage Omega Network (Cont.)

- Machines like IBM eServer p575 and SGI Altix 4000 use Ω -network
- Ω -network is much more interesting for large number of processors
- Multiprocessor vector-processors use instead crossbars (because at most only 32 processors)
- Synchronization is performed with special communication registers (CPU to CPU); if there is little synchronization shared memory is admitted

Shared Memory Interconnection Network (Revisited)

There are three main network topologies available:

- Crossbar ($n \times n$ connections - datapath without sharing)
- Ω -network ($n \log n$ connections - $\log n$ switching stages and shared on a path)
- Central databus ($O(1)$ connections - n shared)

Advantages of Shared Memory Machines

- User-friendly programming environment due to **global address space**
- Data sharing is **fast and uniform** due to proximity (nearness) of memory to CPUs
- Memory coherence is **managed** by the operating system
 - Memory Coherence: Multiple processors trying to access the same memory location

Disadvantages of Shared Memory Machines

- Performance **degradation** due to “memory contention” (several processors try to access the same memory location)
- Programmer’s responsibility for **synchronization** constructs (correct access to memory)
- Expensive to **design** shared memory computers with increasing numbers of processors
- Adding processors can geometrically increase **traffic** on the shared memory-CPU path and for cache coherence management
 - Cache Coherence: Multiple processors trying to access the same cache

Distributed-Memory SIMD

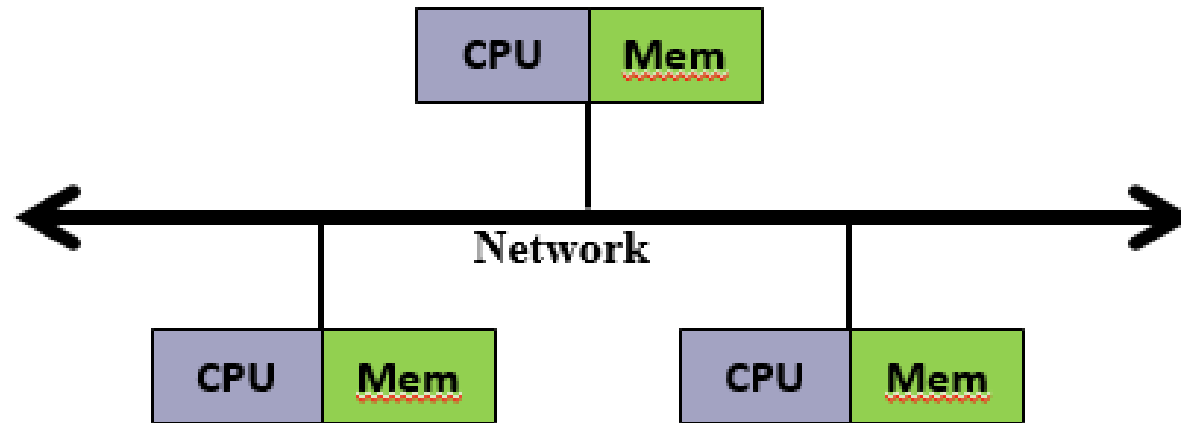
- These machines are sometimes also known as **processor-array** machines
- They work in **lock-step**, i.e. all processors execute the same instruction at the same time (but on different data items); no synchronization is required
- A **control processor** issues the instructions that are to be executed by the processors in the processor array

Distributed-Memory SIMD (Cont.)

- GPUs are **similar** to processor array systems
- DM-SIMD are **specialized** on digital signal and image processing and on certain types of Monte Carlo simulations with virtually no exchange between processors
 - Monte Carlo Simulation is a mathematical technique that is used to estimate **possible outcome** of an uncertain event.
- Operations that **cannot be executed** by the processor array or by the control processor are off-loaded to the front-end system
- I/O may be through:
 - the front end system
 - the processor array
 - both

Distributed-Memory MIMD

- Processors have their own **local** memory
- **No concept** of global address space across all processors
- Distributed memory systems require a communication **network** to connect inter-processor memory



Examples

Shared-memory SIMD systems

- The Hitachi S3600 series

Distributed-memory SIMD systems

- The Alenia Quadrics
- The Cambridge Parallel Processing Gamma II
- The Digital Equipment Corp. MPP series
- The MasPar MP-1
- The MasPar MP-2

Shared-memory MIMD systems

- The Cray Research Inc. Cray J90-series, T90 series
- The Hitachi S3800 series
- The HP/Convex C4 series
- The Digital Equipment Corp. AlphaServer
- The NEC SX-4
- The Silicon Graphics Power Challenge
- The Tera MTA

Examples (Cont.)

Distributed-memory MIMD systems

- The Alex AVX 2
- The Avalon A12
- The C-DAC PARAM 9000/SS
- The Cray Research Inc. T3E
- The Fujitsu AP1000
- The Fujitsu VPP300 series
- The Hitachi SR2201 series
- The HP/Convex Exemplar SPP-1200
- The IBM 9076 SP2

Distributed-memory MIMD systems

- The Intel Paragon XP
- The Matsushita ADENART
- The Meiko Computing Surface 2
- The nCUBE 3
- The NEC Cenju-3
- The Parallel Computing Industries system
- The Parsys TA9000
- The Parsytec GC/Power Plus

DM-MIMD Routing Mechanism

- Routing mechanism determines the **path** a message takes through network to reach from source to destination node.
- **Data and task decomposition** have to be dealt explicitly!
- Topology and speed of the data paths are crucial and have to be **balanced** with costs.
- Routing can be **classified** as:
 - Minimal
 - Non-minimal
- It can also be **classified** as:
 - Deterministic routing
 - Adoptive routing

Linear Arrays

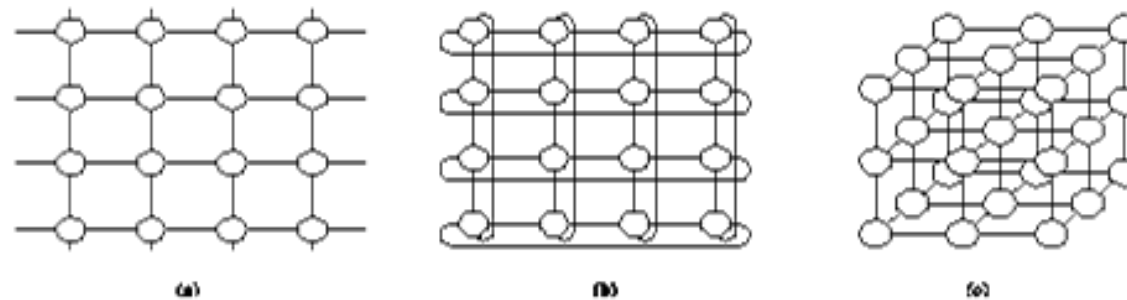
- In a linear array, each node has two **neighbors**, one to its left and one to its right.
- If the nodes at either end are **connected**, we refer to it as a 1-D torus or a ring.



Linear arrays: (a) with no wraparound links; (b) with wraparound link.

K-d Meshes

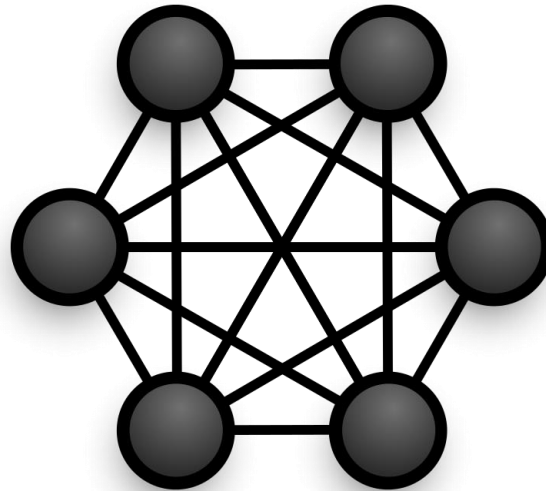
- A generalization has nodes with **4 neighbors**, to the north, south, east, and west.
- A further generalization to d dimensions has nodes with **$2d$ neighbors** (i.e., 6 neighbors in case of 3d cube).
 - 3d-weather modeling, 3d-structure modeling



Two and three dimensional meshes: (a) 2-D mesh with no wraparound; (b) 2-D mesh with wraparound link (2-D torus); and (c) a 3-D mesh with no wraparound.

Star Connected Networks

- Each processor is **connected** to every other processor (Complete connected network)
- The number of links in the network scales as $O(p \times p)$
- While the performance scales very well, the **hardware complexity** is not realizable for large values of p

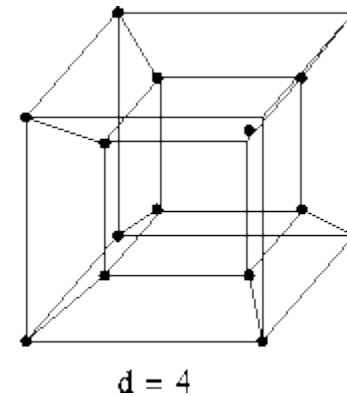
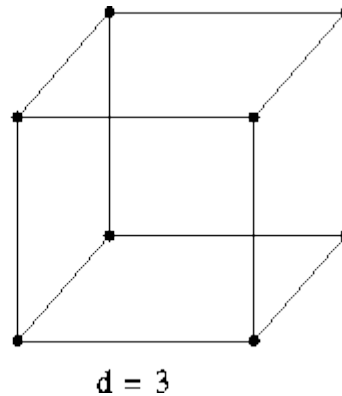
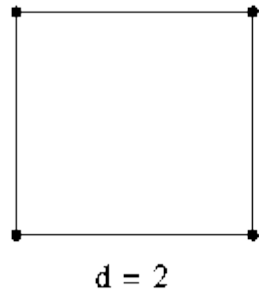
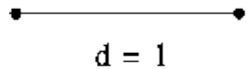


Hypercube

- For an hypercube with 2^d nodes the number of steps to be taken between any two nodes is at most d (logarithmic grow)

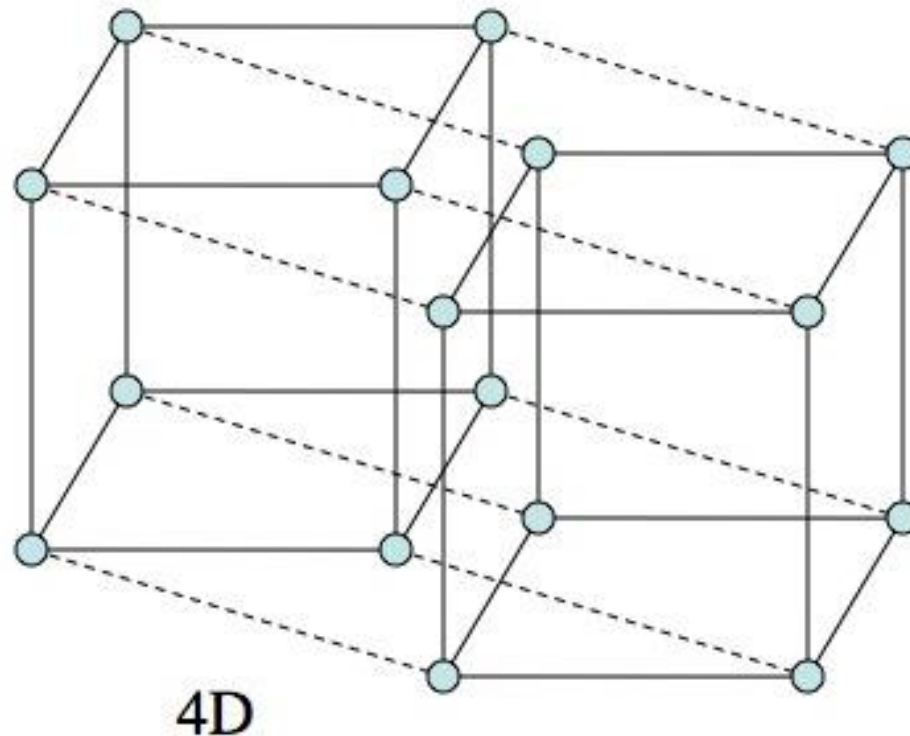
$$d = \log p \text{ (dimensions = } \log(\text{nodes}))$$

- The distance between any two nodes is at most $\log p$.
- Each node has $\log p$ neighbors.



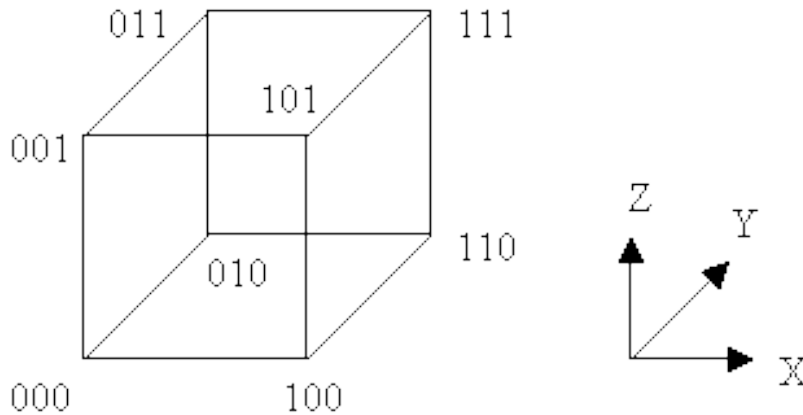
Hypercube (Cont.)

- **Rule of thumb:** “d-dimensional hypercube can be **constructed** by connecting corresponding nodes of two different (d-1)-dimensional hypercubes”



Hypercube (Cont.)

- The processors are numbered with 3-bit binary numbers which represent the X-Y-Z coordinates
- The **distance between two nodes** is given by the number of bit positions at which the two nodes differ.

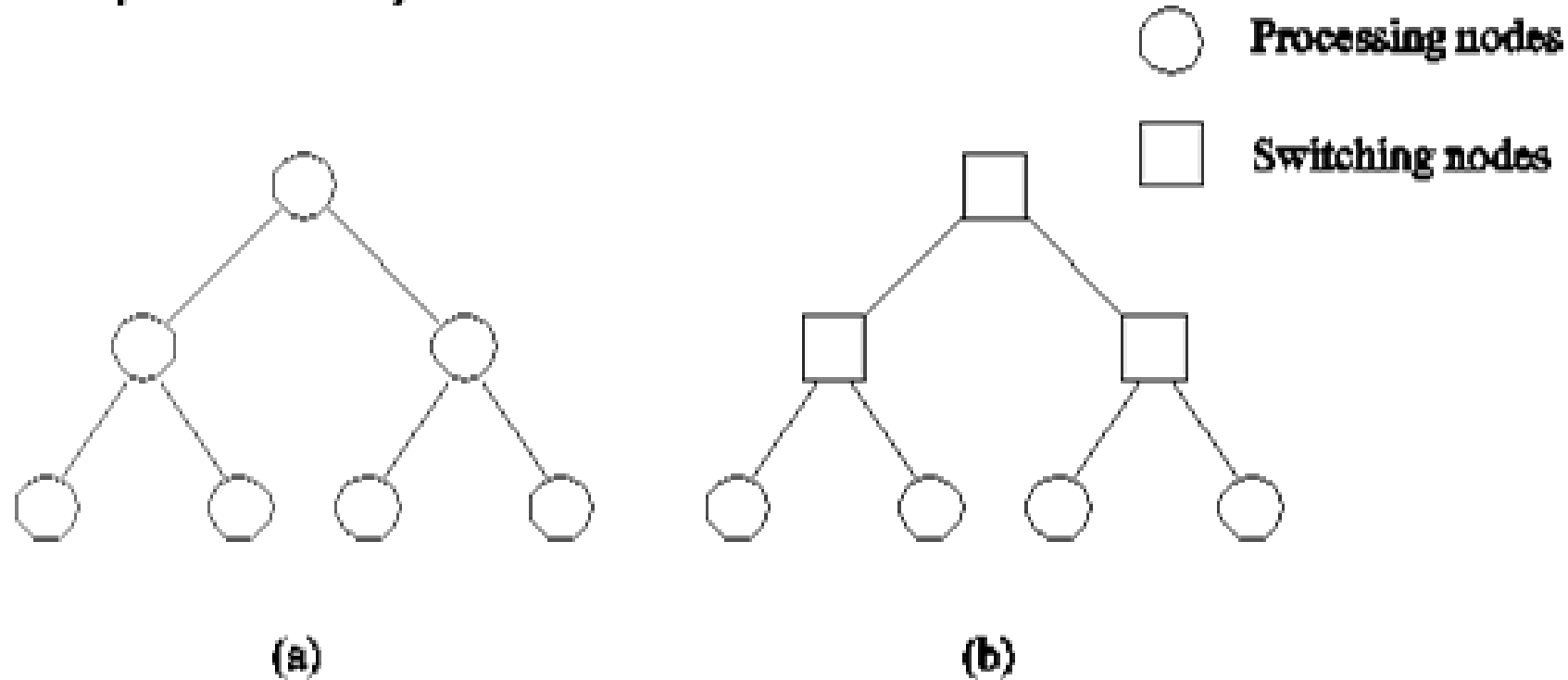


Tree Based Networks

- A tree network is one in which there is **one path** between any pair of nodes
- Linear arrays and star-connected networks are **special cases** of tree-based networks
- In **static** tree network, each node represent a processing element
- In **dynamic** tree network, leaf nodes represent processing element while internal nodes are switching elements

Tree Based Networks (Cont.)

Complete Binary Tree



Complete binary tree networks: (a) a static tree network; and (b) a dynamic tree network.

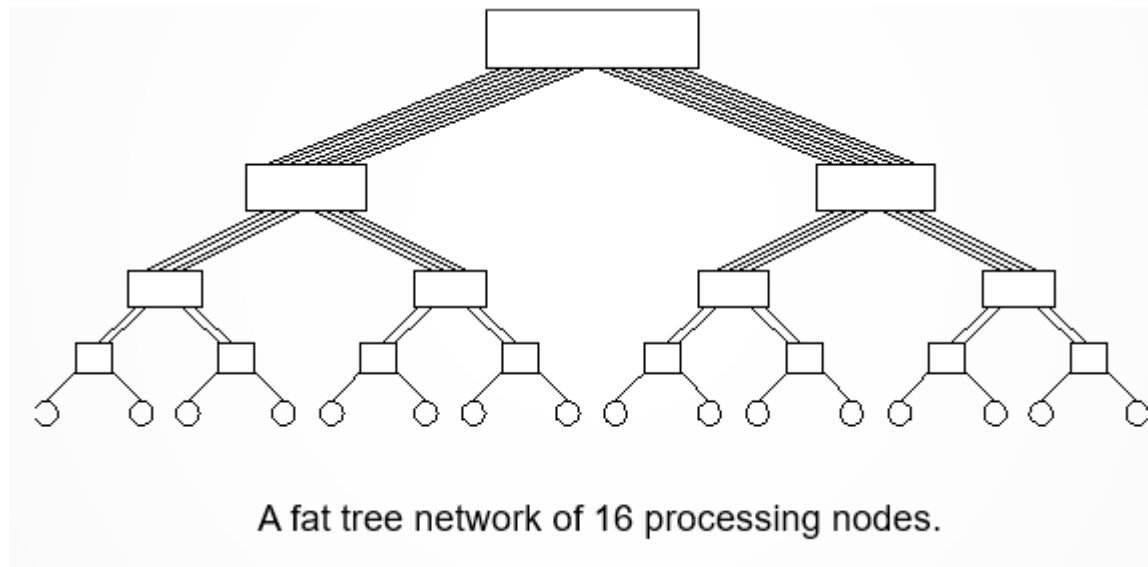
Tree Based Networks (Cont.)

Properties of Complete Binary Tree Network

- The **distance** between any two nodes is no more than $2\log p$.
- Links higher up the tree potentially carry **more traffic** than those at the lower levels.
- For this reason, a variant called a **fat-tree**, fattens the links as we go up the tree.
- Trees can be **laid out** in 2D with no wire crossings. This is an attractive property of trees.

Fat Tree

- In a tree, a node can speak to another node passing through the root so we have **higher traffic** at the root node.
- Fat tree amends this providing **more bandwidth** (with multiple connections) in the higher levels of the tree



Evaluating Static Interconnections

The parameters to evaluate a static interconnection:

- **Cost:** Usually depends on **number of links** for communication. E.g., cost for linear array is **$p-1$** .
 - **Lower** values are favorable
- **Diameter:** The **shortest distance** between the farthest two nodes in the network. The diameter of a linear array is **$p - 1$** .
 - **Lower** values are favorable
- **Bisection Width:** The minimum number of wires you must **cut** to divide the network into two equal parts. The bisection width of a linear array is 1.
 - **Higher** values are favorable

Evaluating Static Interconnections (Cont.)

- **Arc-connectivity:** The minimum number of arcs or links that must be **removed** from the network, to break the network into two disconnected networks
 - **Higher** value are desirable
 - It is minimum number of the links that must be cut to separate the **single node** from the network
 - Higher values means, that incase of link failure there are **multiple other routes** to the node.
 - Arc-connectivity of **linear array** is 1 and 2 for ring.

Evaluating Static Interconnections (Cont.)

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Completely-connected	1	$p^2/4$	$p - 1$	$p(p - 1)/2$
Star	2	1	1	$p - 1$
Complete binary tree	$2 \log((p + 1)/2)$	1	1	$p - 1$
Linear array	$p - 1$	1	1	$p - 1$
2-D mesh, no wraparound	$2(\sqrt{p} - 1)$	$\sqrt{p}$	2	$2(p - \sqrt{p})$
2-D wraparound mesh	$2\lfloor \sqrt{p}/2 \rfloor$	$2\sqrt{p}$	4	$2p$
Hypercube	$\log p$	$p/2$	$\log p$	$(p \log p)/2$

Summary

- **Shared Memory Interconnection Network**

- Crossbar - $n \times n$ connections, no path sharing
- Omega Network - $n \log n$ connections, shared path
- Bus - $O(1)$ connections, shared path

- **Bus**

- Simplest network topology for processor to processor and processor to memory communication
- Excellent cost scalability but poor performance scalability

- **Crossbar Network**

- A $p \times m$ grid of switches to connect p inputs to m outputs in a non-blocking manner
- Excellent performance scalability but poor cost scalability

Summary (Cont.)

- **Multistage Omega Network**
 - Most commonly used multistage interconnects topology
 - $\log p$ stages, input to output decided by perfect shuffle
 - Example: 8 x 8 Omega network
- **Linear Arrays**
- **K-d Meshes**
- **Hypercube**
 - Each node has $\log p$ neighbors

Summary (Cont.)

- **Tree Based Networks**

- In static tree network, each node represent a processing element
- In dynamic tree network, leaf nodes represent processing element while internal nodes are switching elements

- **Fat Tree**

- More bandwidth (with multiple connections) in the higher levels of the tree

Summary (Cont.)

- **Evaluating Static Interconnections**

- **Cost** - Depends on number of links for communication
- **Diameter** - Shortest distance between two most distant nodes
- **Bisection Width** - Minimum number of wires you must cut to divide the network into two equal parts
- **Arc Connectivity** - Minimum number of links that must be removed from the network, to break the network into two disconnected networks

- **Exercise**

- **4 x 4 Omega Network**

Additional Resources

- **Introduction to Parallel Computing** by Ananth Grama and Anshul Gupta

- **Chapter 2: Parallel Programming Platforms**

- ✓ **Section 2.4.3: Network Topologies**
- ✓ **Section 2.6: Routing Mechanisms for Interconnection Networks**

Questions?

Class Activity